

SystemMessage with Recent Conversation Window

1. What About SystemMessage?

Suppose we have:

```
system_message = SystemMessage(  
    content="You are a helpful Python tutor."  
)
```

and conversation history separately:

```
history = []
```

A clean design is:

SystemMessage

+

Recent Conversation

↓

Model

2. Better Architecture

Keep the system instruction separately:

```
system_message = SystemMessage(  
    content="You are a helpful Python tutor."
```

)

Store conversation history separately:

```
history = []
```

Select recent conversation:

```
recent_history = history[-6:]
```

Then prepare the input:

```
input_messages = [  
    system_message,  
    *recent_history  
]
```

This gives us:

SystemMessage

+

Recent History

↓

Model Input

3. Why Keep SystemMessage Separately?

If the SystemMessage is mixed inside the conversation list and later we slice the list, it may disappear from the selected window.

So a cleaner design is:

SystemMessage



Keep Separately

Conversation History



Store Separately

Recent Window



Select from History

Then combine them only when calling the model.

4. Program — SystemMessage + Recent Conversation Window

import os

from langchain_openai import ChatOpenAI

from langchain_core.messages import (

SystemMessage,

```
HumanMessage  
)
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
# System instruction is kept separately  
system_message = SystemMessage(  
    content="""  
    You are a helpful AI assistant.  
    Answer in simple English.  
    """"  
)
```

```
# Complete conversation history
```

```
history = []
```

```
MAX_MESSAGES = 6
```

```
while True:
```

```
    user_input = input(  
        "\nYou: "  
    )
```

```
    if user_input.lower() == "exit":
```

```
        print("Chat ended.")
```

```
        break
```

```
# Add user message
history.append(
    HumanMessage(
        content=user_input
    )
)
```

```
# Select recent conversation
recent_history = history[
    -MAX_MESSAGES:
]
```

```
# Always include system message
input_messages = [
    system_message,
    *recent_history
]
```

```
# Send selected context
response = llm.invoke(
    input_messages
)

print(
    "AI:",
    response.content
)

# Add AI response to complete history
history.append(
    response
)
```

5. Three Separate Concepts

In this program, we maintain three different things.

1. system_message

System Instruction



Always Included

Example:

```
system_message = SystemMessage(  
    content="You are a helpful AI assistant."  
)
```

2. history

Complete Conversation Storage

Example:

```
history = []
```

It contains the complete conversation maintained by the application.

3. recent_history

Recent Conversation



Sent to Model

Example:

```
recent_history = history[-6:]
```

6. Complete Architecture

FULL HISTORY



Select Recent Window



Recent History

+

SystemMessage



Model



AI Response



Add Back to Full History

7. Most Important Difference

history



Complete Stored Conversation

recent_history



Only Recent Conversation

input_messages



SystemMessage + Recent Conversation

So:

Stored History

≠

History Sent to Model

8. Why Use `*recent_history`?

Consider:

```
input_messages = [  
    system_message,  
    *recent_history  
]
```

The `*` operator unpacks all items from `recent_history`.

Suppose:

```
recent_history = [  
    message1,  
    message2,  
    message3  
]
```

Then:

```
input_messages = [  
    system_message,  
    *recent_history  
]
```

becomes conceptually:

```
input_messages = [  
    system_message,  
    message1,  
    message2,  
    message3  
]
```

9. Model Input Structure

Suppose recent history contains:

HumanMessage

AIMessage

HumanMessage

AIMessage

Then the final model input becomes:

SystemMessage

HumanMessage

AIMessage

HumanMessage

AIMessage

So the model receives:

System Instruction

+

Recent Conversation Context

10. Program — Debug Stored Messages vs Sent Messages

This program helps students clearly see the difference between:

Total Stored History

and:

Current Model Context

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import (
```

```
    SystemMessage,
```

```
    HumanMessage
```

```
)
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
system_message = SystemMessage(
```

```
    content="You are a helpful AI assistant."
```

```
)
```

```
history = []
```

```
MAX_MESSAGES = 6
```

```
while True:
```

```
    user_input = input(
        "\nYou: "
    )
```

```
    if user_input.lower() == "exit":
        break
```

```
    history.append(
        HumanMessage(
```

```
        content=user_input
    )
)
```

```
recent_history = history[
    -MAX_MESSAGES:
]
```

```
input_messages = [
    system_message,
    *recent_history
]
```

```
response = llm.invoke(
    input_messages
)
```

```
print(  
    "AI:",  
    response.content  
)
```

```
history.append(  
    response  
)
```

```
print(  
    "\nTotal Stored Messages:",  
    len(history)  
)
```

```
print(  
    "Recent Conversation Messages:",
```

```
len(recent_history)
)

print(
    "Total Messages Sent to Model:",
    len(input_messages)
)
```

11. Understanding the Counts

Suppose:

Total Stored Messages = 20

and:

Recent Conversation Messages = 6

Then:

SystemMessage

+

6 Recent Conversation Messages

=

7 Messages Sent to Model

Therefore:

Total Stored Messages: 20

Recent Conversation Messages: 6

Total Messages Sent to Model: 7

12. Why Is the Final Count 7?

Because:

1 SystemMessage

+

6 Recent Conversation Messages

=

7 Messages

Very important:

MAX_MESSAGES = 6

refers to the recent conversation messages.

The SystemMessage is added separately.

13. Stored History vs Model Context

Suppose:

history = 20 messages

But:

recent_history = history[-6:]

Then:

Stored by Application



20 Messages

But:

Recent Conversation Sent



6 Messages

Plus:

SystemMessage

So:

Total Model Input



7 Messages

14. What Can We Do With Important Old Information?

Later, more advanced designs can use:

1. Summaries
2. Persistent state
3. Separate user profile information
4. Databases
5. Retrieval
6. LangGraph state/checkpoints

For now, just remember that these are possible solutions.

15. Common Mistake 1 — Thinking History Can Grow Forever

Wrong idea:

Python list can grow



So conversation history can be sent forever

Not correct.

Even if Python can store a large list, the model request still has context constraints.

Remember:

Python Storage

≠

Model Context Limit

16. Common Mistake 2 — Thinking history[-6:] Means Six Tokens

Wrong:

history[-6:]

does not mean:

Last 6 Tokens

It means:

Last 6 List Items

In this case:

Last 6 Message Objects

17. Messages vs Tokens

Very important:

Message Count

≠

Token Count

One message may be very short.

Another message may be very long.

Example:

Message 1

"Hi"

Message 2

500-word explanation

Both count as:

1 Message

but token usage is very different.

18. Common Mistake 3 — Messages vs Conversation Turns

Wrong:

3 messages

=

3 conversation turns

Not necessarily.

Normally one conversation turn may contain:

HumanMessage

+

AIMessage

So:

1 Turn



Approximately 2 Messages

Therefore:

6 Messages



Approximately 3 Human + AI Turns

assuming normal alternating conversation.

19. Common Mistake 4 — Trimming the Full List Accidentally

Consider:

```
history = history[-6:]
```

This replaces the current history list with the sliced list.

So old messages are no longer available in that list.

Do not say:

Full history is still available.

after doing this.

20. If You Want to Preserve Full History

Use:

```
recent_history = history[-6:]
```

Now:

history



Full History

and:

recent_history



Recent 6 Messages

remain separate.

21. Difference Between Two Approaches

Approach 1 — Preserve Full History

```
recent_history = history[-6:]
```

Result:

history



Full History Remains

recent_history



Recent Window

Approach 2 — Trim Actual History

`history = history[-6:]`

Result:

`history`



Only Recent 6 Messages Remain

22. Common Mistake 5 — Thinking Smaller Window Is Always Better

Wrong.

A smaller window gives:

Less Context Usage



Fewer Messages Sent

But it can lose:

Important Earlier Information

Example:

Old Message:

"My name is Durga."

Later:

"What is my name?"

If the old message falls outside the window, the model may not receive it.

23. Common Mistake 6 — Thinking Bigger Window Is Always Better

Wrong.

A bigger window may provide more context.

But it can also increase:

Input Size

Token Usage

Cost

Latency

Irrelevant Context

Therefore, there is a trade-off.

24. Window Size Trade-Off

Small Window

SMALL WINDOW



Less Context



Lower Input Size

But:

May Lose Important Old Information

Large Window

LARGE WINDOW



More Context

But:

More Input



More Tokens



Potentially More Cost



Potentially More Latency

25. Important Rule

There is no single perfect window size for every application.

Window size depends on:

Application

Conversation

Model Context Window

Importance of Old Information

Cost Requirements

26. Day 3 Master Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import (
```

```
    SystemMessage,
```

```
    HumanMessage
```

```
)
```

```
# -----
```

```
# Create Chat Model
```

```
# -----
```

```
llm = ChatOpenAI(  
    model="gpt-5.6-luna",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
# -----
```

```
# System Instruction
```

```
# -----
```

```
system_message = SystemMessage(  
    content="""  
    You are a helpful AI assistant.  
    Answer in simple English.  
    """"  
)
```

```
# -----
```

```
# Complete Conversation History
```

```
# -----
```

```
history = []
```

```
# Keep only recent 6 conversation messages
```

```
# when sending to the model
```

```
MAX_MESSAGES = 6
```

```
print("AI Chatbot")
```

```
print("Type 'exit' to stop")
```

```
print("-----")
```

```
# -----
```

```
# Chat Loop
```

```
# -----
```

```
while True:
```

```
    # Get user input
```

```
    user_input = input(  
        "\nYou: "  
    )
```

```
    # Stop chatbot
```

```
    if user_input.lower() == "exit":
```

```
        print("Chat ended.")
```

```
        break
```

```
# Add user message to complete history
```

```
history.append(  
    HumanMessage(  
        content=user_input  
    )  
)
```

```
# Select recent conversation history
```

```
recent_history = history[  
    -MAX_MESSAGES:  
]
```

```
# System instruction
```

```
# +
```

```
# Recent conversation
```

```
input_messages = [  
    system_message,  
    *recent_history  
]
```

```
# Send selected context to model
```

```
response = llm.invoke(  
    input_messages  
)
```

```
# Display response
```

```
print(  
    "AI:",  
    response.content
```

```
)
```

```
# Store AI response in full history
```

```
history.append(  
    response  
)
```

```
# Debug information
```

```
print(  
    "\nTotal Stored Messages:",  
    len(history)  
)
```

```
print(  
    "Recent Conversation Messages Used:",  
    len(recent_history)
```

)

27. Master Program Flow

User Input



Create HumanMessage



Add to Full History



history



Select Recent Window



recent_history



Add SystemMessage



input_messages



Send to Model



AI Response

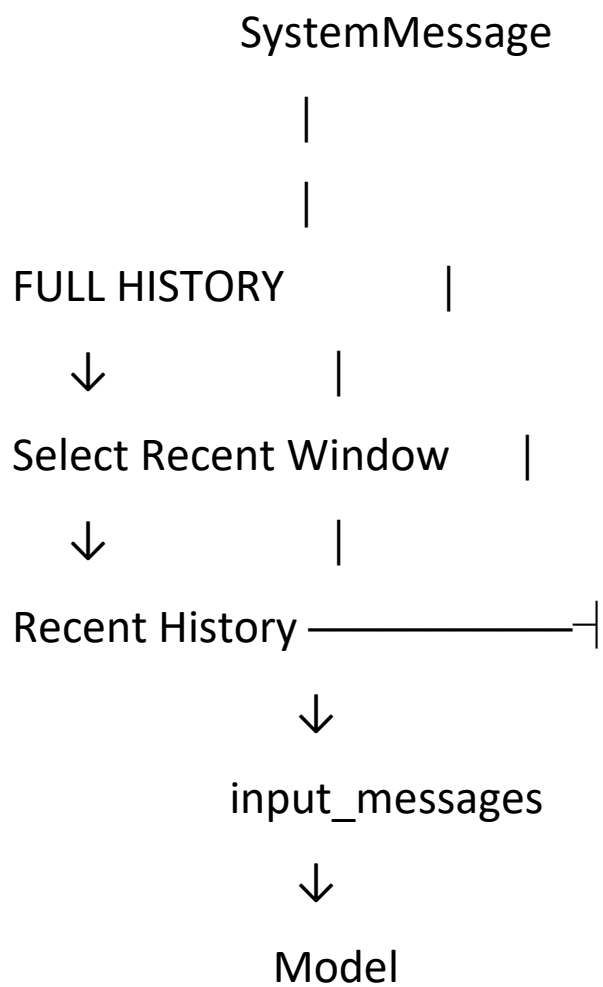


Add Response to Full History



Repeat

28. Complete Architecture





AI Response



Add to Full History

29. Why This Architecture Is Cleaner

We clearly separate:

System Instructions

from:

Conversation Storage

and from:

Current Model Context

So we have:

`system_message`



Permanent instruction for each call

`history`



Complete conversation storage

`recent_history`



Selected recent conversation

input_messages



Actual messages sent to model

30. Important Key Points

At the end of the topic, remember these five points.

1.

Conversation history grows as the chatbot continues.

2.

Conversation history contributes to model input/context usage.

3.

Stored history and history sent to the model do not have to be the same.

4.

A window keeps only recent messages for the current model call.

5.

A small window saves context but may lose important old information.

31. Additional Important Point

Keep the SystemMessage separately and include it whenever sending the recent conversation to the model.

SystemMessage

+

Recent History

↓

Model

32. MCQ 1

What does this mean?

```
recent_history = history[-6:]
```

- A. Last 6 tokens
- B. First 6 messages
- C. Last 6 messages
- D. Delete all history

Answer: C

Conclusion

List slicing selects the most recent six message objects.

33. MCQ 2

If we have:

```
history = [...]
```

```
recent_history = history[-6:]
```

is the complete history automatically deleted?

A. Yes

B. No

Answer: B

Conclusion

We only created another list containing the selected recent messages.

The original history remains available.

34. MCQ 3

Why might a very small conversation window be a problem?

A. It changes the API key

B. It can remove important old information from current model context

C. It retrains the model

D. It disables AIMessage

Answer: B

Conclusion

Recent-only context may exclude useful older information.

35. Master Comparison

Concept	Purpose
system_message	Holds system instruction
history	Stores complete conversation
recent_history	Holds recent conversation window
input_messages	Actual messages sent to model
response	AI response returned by model

36. Easy Memory Formula

SYSTEM MESSAGE

+

RECENT HISTORY

↓

MODEL

while:

FULL HISTORY

↓

Remains Stored Separately

37. Stored vs Selected vs Sent

STORED



history

SELECTED



recent_history

SENT



input_messages

And:

input_messages

=

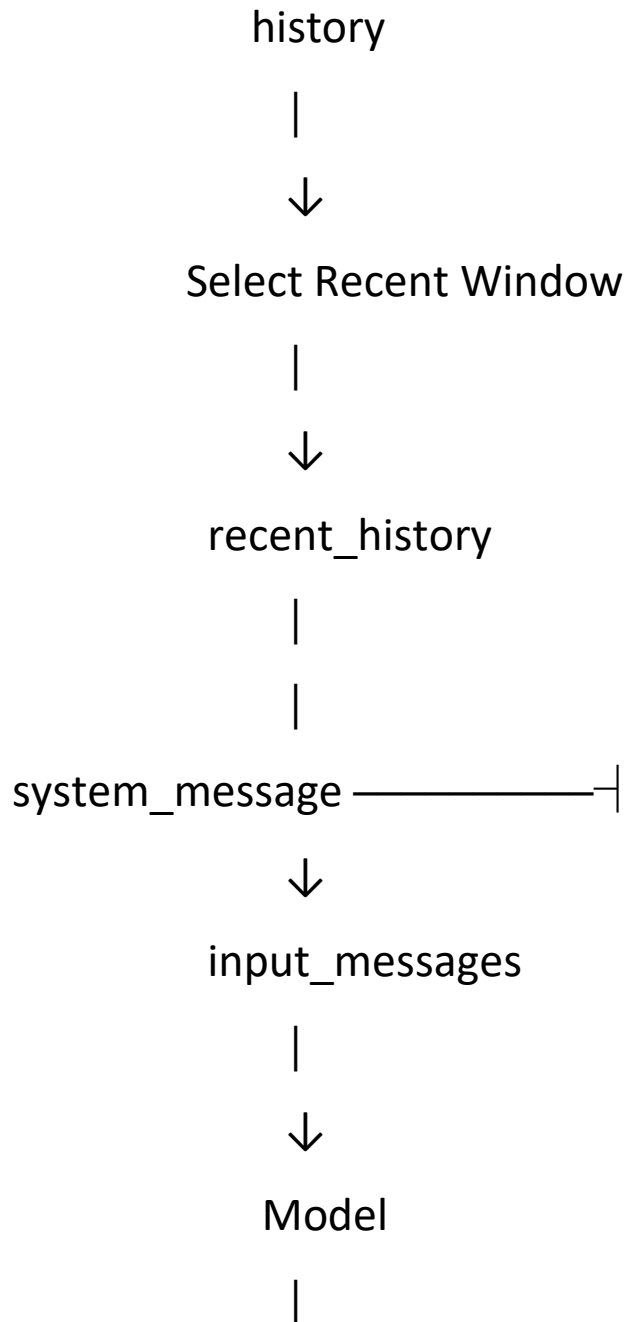
SystemMessage

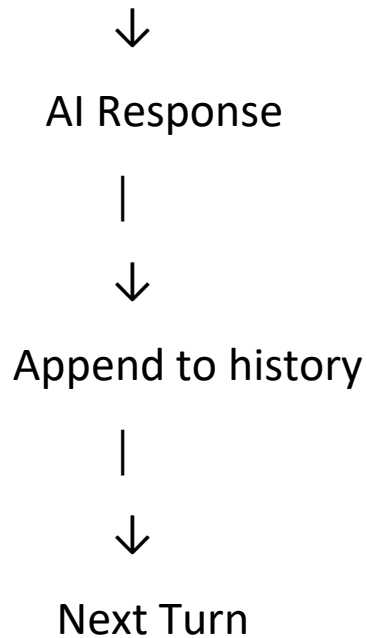
+

recent_history

38. Final Master Diagram

COMPLETE CHAT SYSTEM





39. One-Line Conclusion

Keep the SystemMessage separately, store the complete conversation in history, select only the recent conversation when needed, and send SystemMessage + recent_history to the model.

40. Final Memory Line

Full History



Take Recent Window



Add SystemMessage



Send to Model



Get AI Response



Store Response in Full History



Repeat